

Agents, Workflows, and MCP: What FL-04 Would Need to Become an Agent

Build (Core) — Agent vs Workflow / MCP Explainer

Editing note: Sections 2 and 5 (the FL-04 classification and connector evidence) were drafted using a stated assumption because the specific FL-04 pipeline details and a live MCP connector session were not available when this draft was produced. Both sections are clearly marked below — replace the assumption in Section 2 with your actual pipeline description, and swap Section 5 for real screenshots of your own connector run, before submitting.

1. Workflow vs. Agent, in my own words

Anthropic's engineering write-up on agentic systems draws one core distinction, and it's about who is holding the map. In a **workflow**, a human has already decided the route: the code defines a fixed sequence of steps, and the LLM is called at specific points along that predetermined path, usually to do one well-scoped thing (summarize, classify, extract, draft) before control returns to the code. The developer decides what happens after each step. Nothing about the sequence of steps changes based on what the model decides mid-run.

In an **agent**, the model itself is holding the map. It receives a goal, then decides — turn by turn — which tool to call, what to do with the result, and when the task is actually finished. The 'code path' isn't fixed in advance; it's discovered as the agent goes, using real feedback from its environment (a file it just read, an API response, a test that failed) to decide the next move. A human can still set boundaries — a tool list, a budget, a stopping condition — but within those boundaries, the model is in control of its own process, not just filling in one blank in someone else's script.

A simple gut-check: if you could draw the entire flow as a fixed flowchart before running it once, and the model's job is only to fill in specific boxes, it's a workflow. If the flowchart would have to be redrawn differently almost every run because the model is choosing which boxes to visit and in what order, it's an agent. Most production systems today are workflows, or workflows with one agentic step buried inside them — fully autonomous agents are the exception, not the default, because workflows are cheaper, faster, and easier to trust for well-defined tasks.

2. Classifying my FL-04 pipeline

Assumption used below (FL-04 specifics were not provided): a typical FL-04 backend pipeline receives an inbound request, validates and authenticates it, reads/writes to a database, optionally pushes slow work onto a background queue, and returns a response — with each of these stages wired together in fixed, hand-written code. Replace this paragraph with what your FL-04 pipeline actually does if it differs (e.g., if any stage lets a model decide which tool to call next, that stage pushes it toward 'agent').

Under that description, FL-04 is a **workflow**, not an agent. Every stage — receive request, authenticate, validate, hit the database, enqueue a job, respond — runs in an order I wrote into the code, not an order the model chose at runtime. Even if an LLM call sits inside one of those stages (say, classifying a support ticket, or extracting fields from a document), that call is just one augmented

step in a chain I control; it doesn't get to decide what the pipeline does next, doesn't get to choose which downstream service to call, and doesn't loop on its own until some open-ended goal is satisfied. That's the textbook definition of a workflow: predefined code paths, with the LLM (if present at all) doing one bounded piece of work inside a step I've already drawn on the flowchart.

3. What MCP is

The Model Context Protocol (MCP) is an open standard, published by Anthropic, for connecting an AI application to external data and tools through one common interface instead of a custom integration for every service. The official framing is a USB-C port for AI applications: any MCP-compatible client (Claude, an IDE, a custom app) can plug into any MCP server (Google Drive, GitHub, a local database, a company's internal API) without bespoke glue code for each pairing.

MCP servers expose their capabilities through three primitives:

- **Tools** — executable functions the model can call to take an action or produce a side effect (run a query, create a ticket, edit a file). The model decides when to invoke them, based on typed parameters described in a schema.
- **Resources** — read-only contextual data the client can fetch and hand to the model for grounding (a file's contents, a database record, a log). These are typically application-controlled rather than model-initiated.
- **Prompts** — reusable, parameterized instruction templates that structure a common task (e.g., a standard document-formatting prompt), usually invoked by the user rather than the model.

The practical effect is that 'giving an LLM tools' stops being an N-times-M integration problem (every app wired to every service by hand) and becomes an N-plus-M problem: any MCP client can talk to any MCP server the moment both speak the protocol.

4. What FL-04 would need to become an agent

Turning FL-04 from a workflow into an agent isn't about sprinkling in more LLM calls — it's about handing the model control over the sequencing itself. Concretely, that means:

- **Replace the fixed call graph with a tool-use loop.** Instead of my code deciding 'validate, then write to DB, then enqueue,' the model would receive the incoming request plus a toolbox (validate(), write_record(), enqueue_job(), notify()) and decide at each turn which tool to call next, based on what the previous tool call returned.
- **Give it ground truth to react to.** Right now my pipeline's steps succeed or fail silently into predefined error branches. An agent needs to see real results — a failed validation, a duplicate-key error, a slow downstream service — and decide what to do about it (retry, skip, escalate to a human) rather than following a branch I hard-coded in advance.

- **Add a stopping condition and a budget.** Since the model would now control its own loop, I'd need an explicit termination rule (task complete, max iterations, or a confidence threshold) so it can't spin indefinitely on an ambiguous request.
- **Wrap the actual backend actions as MCP tools** (or an equivalent tool-use interface) with careful docstrings and type constraints — the 'agent-computer interface' the Anthropic essay emphasizes — so the model's choices are constrained to safe, well-documented operations instead of raw code execution.
- **Keep a human checkpoint for anything irreversible** (e.g., actually deleting a record, or issuing a refund), since autonomy should scale with how cheaply a mistake can be undone.

The single concrete upgrade I'd prioritize first: replacing the fixed validate-then-store-then-queue sequence with a small tool-use loop where the model decides, based on the actual shape of the incoming request, which validation and storage steps apply — since the current fixed order is the one part of the pipeline most likely to need a different path depending on the request type, and it's exactly the kind of judgment call an agent handles better than a hard-coded branch.

5. Connector evidence

Status: this draft was produced in a chat session where a Google Drive connector is associated with the account but was not enabled for this specific conversation, so no live tool calls could be executed here. To complete this section for submission: enable the connector for your Claude session (or any MCP client — Claude Desktop, Cursor, an IDE), then run and screenshot three tasks that plain chat could not do, for example: (1) listing files in a specific Drive folder, (2) reading and summarizing the contents of a named document, (3) searching Drive for files matching a keyword and returning the actual matches. Each screenshot should show the tool-call block (not just a text answer) to prove real tool use occurred.